

Završni rad - Tick Box i Chatbot

Završni rad - Optimizacija i LLM-ovi.pdf

PDF

evo ti pdf posljednjeg chata, procitaj ga da si up to date. sljedece sto bih htio implementirati je da ima tick box u kojem oznacis ucis li za MI ili za ZI. takoder me zanima koliko je lagano implementirati chatbota za MVP u streamlitu?

Pročitao sam PDF od zadnjeg chata, i "Only MI / Only ZI" filter je doslovno među predloženim UI/UX upgradeovima koje si već imao u planu.

Završni rad - Optimizacija i LL...

1) Tick box: MI ili ZI (najlakša varijanta)

Ako u CSV-u već imaš stupac `exam_type` s vrijednostima "MI" i "ZI", ovo je 3 linije posla: UI widget + filter prije scoringa/planiranja.

Opcija A (najčišće): radio (odabereš jedno)

```
mode = st.radio("Učim za:", ["MI", "ZI"], horizontal=True)
df_mode = df[df["exam_type"] == mode]
```

Opcija B: checkbox "Samo MI" (inače sve)

```
only_mi = st.checkbox("Samo MI")
df_mode = df[df["exam_type"] == "MI"] if only_mi else df
```

Opcija C: dva checkboxa (može biti i oba)

```
mi = st.checkbox("MI", value=True)
zi = st.checkbox("ZI", value=True)

allowed = []
if mi: allowed.append("MI")
if zi: allowed.append("ZI")

df_mode = df[df["exam_type"].isin(allowed)] if allowed else df.iloc[0:0]
```

Ovo se savršeno uklapa u tvoj postojeći Streamlit MVP pristup gdje se plan računa na svaki rerun.

Završni rad - Optimizacija i LL...

2) Koliko je lagano napraviti chatbota u Streamlitu za MVP?

Iskreno, lagano je za "UI dio": Streamlit ima gotove chat elemente `st.chat_input` i `st.chat_message`, i standardni pattern je držati povijest u `st.session_state["messages"]`. [docs.streamlit.io+2](https://docs.streamlit.io/2)

Što je "teško" ovisi o tome što želiš iza chata:

- **Echo bot (najbrži demo):** 10 minuta.
- **RAG bot nad tvojim PDF-ovima/CSV-om:** srednje, ali izvedivo za završni, jer već imaš jasne izvore (MI/ZI PDF-ovi + indeks).

- **Pravi tutor koji “ne izmišlja zadatke” nego samo vuče postojeće + citira stranicu:** to je najzdraviji MVP, i to si već naglasio kao requirement u projektu.

Minimalni kostur (UI + history) u Streamlitu izgleda ovako (bez backend logike, samo da vidiš koliko je malo):

```
import streamlit as st

st.title("ASP Chat MVP")

if "messages" not in st.session_state:
    st.session_state.messages = [{"role": "assistant", "content": "Reci MI ili ZI i što ti treba."}]

for m in st.session_state.messages:
    with st.chat_message(m["role"]):
        st.markdown(m["content"])

prompt = st.chat_input("Upiši pitanje...")
if prompt:
    st.session_state.messages.append({"role": "user", "content": prompt})
    with st.chat_message("user"):
        st.markdown(prompt)

    # TODO: ovdje zoveš svoju logiku (RAG nad PDF-om, lookup u CSV, itd.)
    answer = f"Primio: {prompt}"

    st.session_state.messages.append({"role": "assistant", "content": answer})
    with st.chat_message("assistant"):
        st.markdown(answer)
```

Za reference, Streamlit dokumentacija baš to pokazuje kao preporučeni način. [docs.streamlit.io+1](https://docs.streamlit.io/)

Moj “no bullshit” savjet za tvoj slučaj

Za MVP, nemoj raditi “general chat”. Napravi chat koji prvo traži mode (MI ili ZI), pa onda radi samo 2 stvari:

1. pronađe relevantne zadatke iz tvog indeksa,
2. otvori preview točne stranice iz odgovarajućeg PDF-a.

To je dovoljno da izgleda kao ozbiljan sustav, i maksimalno je “safe” protiv halucinacija.

Ako želiš, mogu ti odmah napisati konkretan `app.py` diff: dodaš MI/ZI toggle, filtriraš `df` prije `build_plan`, i (po želji) istu tu varijablu koristiš da chatbot zna iz kojeg PDF-a smije citirati.

Završni rad - Optimizacija i LL...

Završni rad - Optimizacija i LL...

Sources

a koliko je tesko da napravim bota koji objasnjava rjesenje na osnovu vec postojećih. On dakle ima rjesenje i onda objasni zasto je to tako, i kako se to sablonizira. To bi bilo izvrsno jer, primjerice, ti meni mozes objasniti kako radi quick sort, ali opet mi je isplativije da odem na predavanje i pogledam točne korake koje je profesor koristio zato sto ce mi on isto ocjenjivati. Zato je bitno da on ima rjesenje kad retrieva to. Mozda mogu spinati neki model lokalno da ne trosim tokene, iako ce biti slabiji. Javi sto mislis o ideji.

To je dobra ideja i, u praksi, puno izvedivija od "generiraj mi novi zadatak". Razlog: ti već imaš ground truth (zadatak + rješenje + izvor iz FER materijala), pa bot ne mora biti kreativan nego mora biti dosljedan.

Koliko je teško

MVP verzija (brzo i realno): srednje lako.

Ako rješenje već postoji u tekstu (ili ga možeš pouzdano izvući iz PDF-a), onda je "objasni zašto" uglavnom prompt + format + par guardrailova.

Verzija koja zvuči kao profesor i prati točno korake s predavanja: srednje teško.

Ovdje je ključ: nije dovoljno da model zna općeniti quicksort, nego mora citirati ili parafrazirati "korake" i definicije iz tvojih slideova/skripte, i to redom.

Verzija koja je 95 posto konzistentna s profesorovim stilom bodovanja: teško, ali dostižno.

To traži ili dodatne "rubric" podatke (što se priznaje kao korak, kako se piše, tipične greške), ili iteraciju kroz stvarne ispite i rješenja.

Kako bih to "sablونizirao" da bude kao predavanje, ne kao opći internet

Najbolji trik: nemoj tražiti od bota da "razmišlja" iz nule. Neka radi kao komentator nad rješenjem, uz obavezne citate iz materijala.

Pipeline koji radi

1. Retrieve "paket":

- tekst zadatka
- službeno rješenje (ili rješenje iz baze)
- relevantan izvadak iz predavanja/skripte (2 do 6 kratkih chunkova) koji definira algoritam ili postupak

2. Model dobije strogu ulogu:

- "Ne smiješ uvoditi korake koji nisu u rješenju."
- "Svaki korak mora referencirati dio rješenja ili dio materijala."
- "Ako nešto nije pokriveno materijalom, napiši da nije eksplicitno navedeno."

3. Output format koji se ponavlja svaki put:

- Kratka ideja (1 do 2 rečenice)
- Ulazi i pretpostavke (što je zadano, što se traži)
- Koraci 1..n (svaki korak: što radimo, zašto, gdje je to u rješenju)
- "Šablona" za slične zadatke (kako prepoznati, koje formule, koje provjere)
- Najčešće greške (ove možeš izvući iz više rješenja i materijala)

To je zapravo “explain this proof”, ali za zadatke.

Zašto ovo radi za quicksort primjer koji si naveo

Ako iz predavanja imaš “točan redoslijed koraka” za partition, izbor pivota, invarijante, složenost, onda model samo mapira rješenje na te korake i ubacuje terminologiju kakvu profesor koristi. Time dobiješ “predavanje vibe” bez finetunea.

Lokalni model da ne trošiš tokene

Može, i često je dovoljno, ali očekuj ovo:

- **Za čistu “parafriziraj rješenje u korake” zadaću**, lokalni model je ok.
- **Za strogu konzistentnost i citiranje**, lokalni model može biti slabiji u “discipliniranosti”, ali to se može popraviti guardrailovima: kraći kontekst, jasna struktura, i re-ranking chunkova da mu ne daš šum.
- **Najveći rizik** je da lokalni model krene “dodavati” objašnjenja koja zvuče pametno, ali nisu iz materijala. To se rješava tako da mu eksplicitno kažeš da je dopušteno samo ono što je u priloženim izvorima, i da koristi citate ili reference tipa “(slajd 23)” ili “(rješenje, korak 4)”.

Ako ti je cilj MVP i wow efekt, ja bih išao ovako:

- lokalno za većinu upita (jeftino),
- opcija “High accuracy” koja koristi jači model samo kad user klikne, ili kad lokalni model padne na provjeri.

Što mislim o ideji, “no bullshit”

Ovo je jedna od rijetkih edukacijskih LLM ideja koja ima smisla, jer:

- ne izmišljaš zadatke,
- imaš reference,
- možeš mjeriti kvalitetu (usporedi s rješenjem i s materijalima),
- value proposition je jasna: “objašnjenje u profesorovom stilu, za točno ovaj zadatak”.

Najbolji sljedeći korak, bez puno filozofije:

Implementiraj “Explain” gumb uz svaki retrieved zadatak. On pošalje u model: (zadatak + rješenje + 3 do 5 chunkova iz predavanja). Output u strogoj strukturi.

Ako zapneš na jednoj stvari, to će biti ekstrakcija rješenja iz PDF-a u čist tekst. Ako su rješenja već u tvojoj bazi kao tekst, onda si praktički već na pola puta.

Ma nije bitno da objasnjava kao profesor, ako ista bolje bi bilo da objasnjava na dumbed down način. U biti sam ciljao na primjer kako, iako AI razumije quick sort bez da mu se da rjesenje, student mozda nece dobiti maksimalne bodove cak i ako napise točno AI rjesenje jer su koraci za quick sort u kolegiju malkice drukčiji od rjesenja koje je predložio AI. Zato bi bilo dobro da ima

rjesenje i onda ga samo treba objasniti. Koliko je tako nesto tesko napraviti? Bilo bi lijepo da je UI totalno jednostavan. Student ako ne razumije zadatak, moze samo kliknuti: "Ne razumijem", i odma mu chatbot objasni zadatak.

Ovo što opisuješ je zapravo "explain the given solution", ne "solve the problem". To je daleko lakše i puno sigurnije za MVP.

Koliko je teško

Ako rješenje već imaš u tekstualnom obliku (ili ga možeš pouzdano izvući), onda je ovo **lagano do srednje lagano**:

- **UI dio ("Ne razumijem" gumb + chat poruka): lagano.**
- **Logika: dohvat zadatak + rješenje + par relevantnih definicija iz materijala, pa generiraj objašnjenje: lagano.**
- **Najteži dio je kvaliteta inputa**, ne AI: kako čisto i konzistentno spremi rješenja, i kako točno povezati "ovo rješenje pripada ovom zadatku".

Ako rješenje nije u tekstu nego je samo slika/skan u PDF-u, onda postaje **srednje teško**, jer moraš riješiti ekstrakciju. Ali i tad možeš za MVP prebaciti rješenja ručno u bazu za top N zadataka.

Kako to izgleda u najjednostavnijem UI-u

Flow koji si rekao je dobar:

1. student vidi zadatak
2. klikne "**Ne razumijem**"
3. app pošalje botu kontekst i bot vrati objašnjenje

To "Ne razumijem" doslovno može biti samo shortcut za prompt tipa: "Objasni ovo rješenje jednostavno, korak po korak, bez preskakanja."

U istom trenutku možeš imati i drugi gumb:

- "Daj intuiciju"
- "Daj postupak kao checklistu"
- "Gdje ljudi najčešće griješe"

Ali MVP ti može biti samo jedan gumb.

Šablona koja radi i sprječava "AI rješenje" problem

Poanta je: bot ne smije izmišljati svoje korake, nego mora objasniti postojeće.

Najstabilniji prompt style je:

- Input: **zadatak, službeno rješenje**, opcionalno **kratki chunk iz materijala** (definicije)

- Output format:
 1. "Što se traži" (1 do 2 rečenice)
 2. "Što rješenje radi" (high level)
 3. Koraci 1..n: svaki korak mora mapirati na dio rješenja
 4. Mini šablona: "Kad vidiš ovakvu vrstu zadatka, radi ovo..."
 5. Brza provjera: "Kako znaš da je rezultat točan"

Ako želiš još više kontrole: natjeraš model da prvo napravi "outline" koraka iz rješenja, pa tek onda objašnjava. To ekstremno smanjuje šansu da ode u svoju verziju quicksorta.

Implementacijski minimalizam

U Streamlitu, ovo je najjednostavniji pattern:

- Kad se prikaže zadatak, držiš u session state:
 - `current_task_text`
 - `current_solution_text`
 - `current_source` (PDF/page ili ID)
- Klik gumba samo ubaci poruku u chat "Ne razumijem" i okine generiranje odgovora na temelju ta tri polja.

To je par desetaka linija.

Realni "gotcha" problemi (da znaš gdje ćeš zapeti)

1. Rješenje mora biti dobro segmentirano.

Ako je rješenje jedan veliki blok bez koraka, model će svejedno objasniti, ali kvaliteta je slabija.

Rješenje: pre-formatiraj rješenja u strukturu (koraci, formule, zaključak). To možeš automatizirati kasnije.

2. Neke stvari u rješenju su implicitne.

Student pita "zašto baš ovo", a u rješenju nema eksplicitnog "zašto". Rješenje: bot smije objašnjavati intuiciju, ali mora jasno reći kad je to "intuicija", a kad je "iz rješenja/materijala".

3. Ako nemaš dobar retrieval za definicije, bot će nekad objasniti "dumbed down" ali krivim pojmovima.

Rješenje: uz rješenje uvijek pošalji 1 do 3 definicijska chunk-a iz predavanja.

Zaključak

Ako ti je cilj: "klikne Ne razumijem, dobije dumbed down objašnjenje službenog rješenja", to je **baš MVP-friendly**. Najteže je imati rješenja u čistom tekstu i pravilno ih povezati sa zadacima. Sam chatbot dio je relativno straightforward.

Ako hoćeš, mogu ti odmah napisati konkretan Streamlit UI snippet za karticu zadatka koja ima "Ne razumijem" gumb i koja automatski gura kontekst u chat odgovor, bez ikakvog dodatnog UI-a.

Okej, shvacam, ali kako spajamo chat bota sad? Prvi put radim pa ne znam, pogotovo u Streamlitu slucaju.

U Streamlitu “spajanje chat bota” je zapravo samo 3 stvari:

1. **UI za chat** (render povijesti + input)
2. **session_state** da pamti razgovor
3. **funkcija koja zove LLM** i vrati odgovor (ili lokalni model)

I onda tvoj “Ne razumijem” gumb samo ubaci user poruku u chat i pozove istu tu funkciju.

Ispod je kompletan, minimalan primjer koji možeš copy pasteati u `app.py`. Ne koristi nikakvu magiju, samo Streamlit chat API i jedan stub za LLM poziv.

```
import streamlit as st

st.set_page_config(page_title="MI/ZI Tutor MVP", layout="wide")
st.title("MI/ZI Tutor MVP")

# ----- 1) State -----
if "messages" not in st.session_state:
    st.session_state.messages = [
        {"role": "assistant", "content": "Klikni 'Ne razumijem' ili napiši pitanje."}
    ]

# Ovo glumi "trenutno odabrani zadatak" koji si već prikazao u UI-u
if "current_task" not in st.session_state:
    st.session_state.current_task = {
        "id": "MI-2025-11-28-1",
        "exam_type": "MI",
        "task_text": "Zadatak: Odredi vrijeme izvođenja u  $\Theta$  notaciji za  $f(n)$ ...",
        "solution_text": "Rješenje: Analiziramo petlju... zaključak je  $\Theta(n \log n)$ .",
        "source": "MI_svi_do_2026.pdf, str. 1"
    }

# ----- 2) LLM call (zamijeni stvarnim) -----
def call_llm(messages, task_ctx):
    """
    messages: chat history (role/content)
    task_ctx: dict sa task_text/solution_text/source
    """
    # MVP: napravi jedan prompt od konteksta + zadnje poruke
    last_user = next((m["content"] for m in reversed(messages) if m["role"] == "user"), "")

    system = f"""
    Ti si tutor koji objašnjava zadatke isključivo na temelju priloženog rješenja.
    Piši jednostavno i korak-po-korak. Ne uvodi alternativne metode.
    Ako nešto nije u rješenju, reci da nije eksplicitno navedeno.

    IZVOR: {task_ctx["source"]}

    ZADATAK:
    {task_ctx["task_text"]}

    SLUŽBENO RJEŠENJE:
    {task_ctx["solution_text"]}
    """.strip()

    # Ovdje bi inače zvao OpenAI/local model. Za sada vraćamo demo odgovor.
    demo_answer = f"""Evo objašnjenja (na temelju službenog rješenja):

    1) Što se traži: treba odrediti složenost.
    2) Što rješenje radi: gleda koliko se puta izvode ključne petlje/pozivi.
    3) Ključna ideja: rješenje dolazi do zaključka  $\Theta(n \log n)$  jer ...

    Pitao si: "{last_user}"
    Ako želiš, mogu razbiti na još sitnije korake ili objasniti svaki red rješenja."""
    return demo_answer

# ----- 3) Layout: lijevo zadatak + gumbi, desno chat -----
left, right = st.columns([1, 1])

with left:
    st.subheader(f"Trenutni zadatak ({st.session_state.current_task['exam_type']})")
    st.markdown(st.session_state.current_task["task_text"])
    with st.expander("Prikaži službeno rješenje"):
        st.markdown(st.session_state.current_task["solution_text"])
        st.caption(st.session_state.current_task["source"])

    # Gumb koji "spaja" zadatak i chat: samo ubaci poruku i okini odgovor
    if st.button("Ne razumijem"):
        st.session_state.messages.append({"role": "user", "content": "Ne razumijem. Objasni rješenje"})
        answer = call_llm(st.session_state.messages, st.session_state.current_task)
        st.session_state.messages.append({"role": "assistant", "content": answer})
        st.rerun()

with right:
    st.subheader("Chat")
    # Chat area (messages list and input)

```



```
# render povijesti
for m in st.session_state.messages:
    with st.chat_message(m["role"]):
        st.markdown(m["content"])

# normalan input
user_prompt = st.chat_input("Pitaj nešto o zadatku...")
if user_prompt:
    st.session_state.messages.append({"role": "user", "content": user_prompt})
    answer = call_llm(st.session_state.messages, st.session_state.current_task)
    st.session_state.messages.append({"role": "assistant", "content": answer})
    st.rerun()
```

Što točno ovdje znači “spojili smo chat bota”

- Chat ima jednu funkciju `call_llm(...)` koja uvijek dobije:
 - chat povijest
 - kontekst trenutnog zadatka (zadatak + rješenje + izvor)
- “Ne razumijem” gumb ne radi ništa posebno, samo:
 - doda user poruku
 - pozove `call_llm`
 - doda odgovor u povijest

To je cijeli “integration”.

Kako to spojiš s pravim LLM-om

Imaš dvije opcije:

Opcija 1: OpenAI API (najlakše)

U `call_llm` umjesto demo stringa pozoveš API. Tu ti ne mogu pogoditi točan snippet bez toga koristiš li Responses API ili Chat Completions i kako si već organizirao ključeve, ali koncept je uvijek isti: `system + messages`.

Opcija 2: Lokalni model (bez tokena, više setupa)

Tada `call_llm` zove lokalni server (npr. Ollama ili LM Studio) HTTP-om i dobije tekst. Streamlit tu nema problema, samo je poziv drugačiji.

Bitna MVP disciplina

Ako želiš izbjeći quicksort “AI varijantu”, nemoj mu davati slobodu:

- uvijek pošalji rješenje,
- reci “ne uvodi alternativne metode”,
- i formatiraj output u korake.

Ako želiš, mogu ti odmah dati dvije varijante `call_llm`: jednu za OpenAI, jednu za Ollama, obje u istom fileu s feature-flagom tipa `USE_LOCAL = True`.

uuu bravo moze. kako cu ovaj file implementirati u onaj prvi btw?

Super. Najjednostavnije je ovo tretirati kao modul koji “uštekaš” u postojeći Streamlit app, umjesto da sve držiš u jednom ogromnom `app.py`.

Ispod imaš 2 pristupa, oba su normalna i MVP-friendly.

Pristup A: Sve ostaje u `app.py`, samo ubaciš “chat blok” u postojeći layout

Ako već imaš “prvi file” koji radi retrieval i prikazuje zadatke, onda samo:

1. Na početku filea dodaš session state za chat (`messages`) i `current_task`.
2. Tamo gdje renderaš zadatak, dodaš gumb “Ne razumijem”.
3. Na desnoj strani (ili ispod) renderaš chat history + input.
4. Poziv modela ide kroz jednu funkciju `call_llm(messages, task_ctx)`.

To je doslovno “copy paste” blokova iz primjera, ali pazi da `current_task` u session stateu uvijek pokazuje na zadatak koji je trenutno odabran u tvojoj aplikaciji.

Pristup B: Napraviš mali modul `chat_ui.py` i import-aš ga u “prvi file” (preporuka)

Ovo je čišće, jer onda `app.py` samo slaže UI, a chat logika je izolirana.

1) Struktura foldera

Primjer:

```
project/
  app.py
  chat_ui.py
  llm_backends.py
```

2) `llm_backends.py` (OpenAI ili Ollama, feature flag)

Ovo je minimalno, bez puno filozofije.

```
import os
import requests

USE_LOCAL = os.getenv("USE_LOCAL", "1") == "1" # 1 = Ollama, 0 = OpenAI

def build_system_prompt(task_ctx: dict) -> str:
    return """
    Ti si tutor koji objašnjava zadatke isključivo na temelju priloženog rješenja.
    Piši jednostavno i korak-po-korak. Ne uvodi alternativne metode.
    Ako nešto nije u rješenju, reci da nije eksplicitno navedeno.

    IZVOR: {task_ctx.get("source", "")}

    ZADATAK:
    {task_ctx.get("task_text", "")}

    SLUŽBENO RJEŠENJE:
    {task_ctx.get("solution_text", "")}
    """.strip()

def call_ollama(messages: list[dict], task_ctx: dict) -> str:
    """
    Za Ollama: pokreni lokalno npr. `ollama serve` i imaj model tipa llama3.1.
    """
    model = os.getenv("OLLAMA_MODEL", "llama3.1")
    url = os.getenv("OLLAMA_URL", "http://localhost:11434/api/chat")

    system = build_system_prompt(task_ctx)

    payload = {
        "model": model,
        "messages": [{"role": "system", "content": system}] + messages,
        "stream": False,
    }
    r = requests.post(url, json=payload, timeout=120)
    r.raise_for_status()
    data = r.json()
    return data["message"]["content"]

def call_openai(messages: list[dict], task_ctx: dict) -> str:
    """
    Za OpenAI: treba OPENAI_API_KEY u env var.
    Koristi official SDK: `pip install openai`.
    """
    from openai import OpenAI

    client = OpenAI()
    system = build_system_prompt(task_ctx)

    # Ovo koristi Chat Completions stil. Ako koristiš Responses API, lako prilagodimo.
    resp = client.chat.completions.create(
        model=os.getenv("OPENAI_MODEL", "gpt-4.1-mini"),
        messages=[{"role": "system", "content": system}] + messages,
        temperature=0.2,
    )
    return resp.choices[0].message.content

def call_llm(messages: list[dict], task_ctx: dict) -> str:
    return call_ollama(messages, task_ctx) if USE_LOCAL else call_openai(messages, task_ctx)
```

3) chat_ui.py (render chata + "Ne razumijem" helper)

```
import streamlit as st
from llm_backends import call_llm

def ensure_chat_state():
    if "messages" not in st.session_state:
        st.session_state.messages = [
            {"role": "assistant", "content": "Klikni 'Ne razumijem' ili napiši pitanje."}
        ]

def push_and_answer(user_text: str, task_ctx: dict):
    ensure_chat_state()
    st.session_state.messages.append({"role": "user", "content": user_text})
    answer = call_llm(st.session_state.messages, task_ctx)
    st.session_state.messages.append({"role": "assistant", "content": answer})

def render_chat(task_ctx: dict):
    ensure_chat_state()

    for m in st.session_state.messages:
        with st.chat_message(m["role"]):
            st.markdown(m["content"])

    user_prompt = st.chat_input("Pitaj nešto o zadatku...")
    if user_prompt:
        push_and_answer(user_prompt, task_ctx)
        st.rerun()
```

4) U tvom “prvom fileu” (app.py) samo pozoveš to gdje treba

Pretpostavimo da ti već imaš odabrani zadatak kao dict `selected_task` koji sadrži tekst zadatka i rješenje.

Minimalni primjer integracije:

```
import streamlit as st
from chat_ui import render_chat, push_and_answer

st.set_page_config(page_title="MVP", layout="wide")

# Ovo je primjer, ti ovo već imaš kroz retrieval/odabir
selected_task = {
    "id": "MI-2025-11-28-1",
    "exam_type": "MI",
    "task_text": "Zadatak: ...",
    "solution_text": "Rješenje: ...",
    "source": "MI_svi_do_2026.pdf, str. 1"
}

left, right = st.columns([1, 1])

with left:
    st.subheader("Zadatak")
    st.markdown(selected_task["task_text"])
    with st.expander("Rješenje"):
        st.markdown(selected_task["solution_text"])
        st.caption(selected_task["source"])

    if st.button("Ne razumijem"):
        push_and_answer("Ne razumijem. Objasni rješenje korak po korak, jednostavno.", selected_task)
        st.rerun()

with right:
    st.subheader("Chat")
    render_chat(selected_task)
```

To je to. Ti si “implementirao chat bot u prvi file”.

Kako to spojiti s tvojim postojećim “odaberi MI/ZI i retrieve zadatak” flow-om

Ključ je da `selected_task` bude baš ono što već prikazuješ studentu. Znači, kad student klikne zadatak iz liste, ti već znaš:

- tekst zadatka
- rješenje
- izvor (pdf, page)
- tip (MI/ZI)

Samo to upakiraš u dict i proslijediš `render_chat(selected_task)`.

Kako pokretati lokalno ili preko OpenAI bez diranja koda

Postavi env var:

Lokalno (Ollama):

- `USE_LOCAL=1`
- `OLLAMA_MODEL=llama3.1` (ili koji već imaš)
- `OLLAMA_URL=http://localhost:11434/api/chat`

OpenAI:

- `USE_LOCAL=0`
- `OPENAI_API_KEY=...`
- `OPENAI_MODEL=gpt-4.1-mini` (ili što želiš)

Na Windowsu (PowerShell) primjer:

```
$env:USE_LOCAL="1"
$env:OLLAMA_MODEL="llama3.1"
streamlit run app.py
```

Jedna praktična napomena za MVP stabilnost

Kad user klikne novi zadatak, ja bih resetirao chat history, da se ne miješaju objašnjenja:

```
if st.session_state.get("last_task_id") != selected_task["id"]:
    st.session_state.last_task_id = selected_task["id"]
    st.session_state.messages = [{"role": "assistant", "content": "Spreman. Pitaj što nije jasno."}]
```

Ako mi zalijepiš mali komad svog "prvog filea" (dio gdje biraš zadatak i gdje ga prikazuješ), mogu ti precizno pokazati gdje se ubacuju ove 3 linije i kako točno složiti `selected_task` iz tvojih postojećih varijabli.

mozes li mi dati kod koji nije hardkodiran nego koji zapravo radi.

Da. Evo “stvarni” kod koji radi end to end, bez hardkodiranih zadataka, pod uvjetom da imaš tablicu zadataka (CSV) gdje je svaki red jedan zadatak i imaš tekst zadatka + tekst rješenja + metadata.

Ispod su 3 filea koje staviš u isti folder. Radi i s lokalnim modelom (Ollama) i s OpenAI, preko env var.

0) Priprema podataka (CSV)

Napravit ćeš `data/tasks.csv` s ovim kolonama (minimalno):

- `id` (unikatan, string)
- `exam_type` (MI ili ZI)
- `task_text` (tekst zadatka)
- `solution_text` (tekst rješenja)
- `source_pdf` (npr. `MI_svi_do_2026.pdf`)
- `page` (int, stranica ili broj stranice, kako god ti vodiš)

Primjer headera:

`id,exam_type,task_text,solution_text,source_pdf,page`

Ako ti rješenja trenutno nisu u tekstu, MVP-friendly opcija je: ručno upiši za top 30 zadataka, kasnije automatiziraš ekstrakciju.

1) `llm_backends.py`

```

import os
import requests
from typing import List, Dict

def _build_system_prompt(task_ctx: Dict) -> str:
    source_pdf = task_ctx.get("source_pdf", "")
    page = task_ctx.get("page", "")
    exam_type = task_ctx.get("exam_type", "")

    return f"""
    Ti si tutor koji objašnjava zadatke ISKLJUČIVO na temelju priloženog zadatka i službenog rješenja.
    Piši jednostavno, "dumbed down", korak po korak, bez preskakanja.
    Ne uvodi alternativne metode, trikove, ni drugačije varijante algoritma.
    Svaki korak mora biti povezan s dijelom rješenja. Ako nešto nije u rješenju, reci da nije eksplicitno

    KONTEKST:
    - Predmet: {exam_type}
    - Izvor: {source_pdf}, str. {page}

    ZADATAK:
    {task_ctx.get("task_text", "")}

    SLUŽBENO RJEŠENJE:
    {task_ctx.get("solution_text", "")}

    FORMAT ODGOVORA:
    1) Što se traži
    2) Što rješenje radi (kratko)
    3) Koraci objašnjenja (1..n)
    4) Šablona za slične zadatke (checklista)
    5) Brza provjera rezultata
    """.strip()

def call_ollama(messages: List[Dict], task_ctx: Dict) -> str:
    """
    Lokalni backend: Ollama.
    Pokreni: 'ollama serve' i imaj model, npr. 'ollama pull llama3.1'.
    """
    model = os.getenv("OLLAMA_MODEL", "llama3.1")
    url = os.getenv("OLLAMA_URL", "http://localhost:11434/api/chat")

    system = _build_system_prompt(task_ctx)
    payload = {
        "model": model,
        "messages": [{"role": "system", "content": system}] + messages,
        "stream": False,
        "options": {
            "temperature": float(os.getenv("OLLAMA_TEMPERATURE", "0.2")),
        }
    }

    r = requests.post(url, json=payload, timeout=180)
    r.raise_for_status()
    data = r.json()
    return data["message"]["content"]

def call_openai(messages: List[Dict], task_ctx: Dict) -> str:
    """
    OpenAI backend.
    Env: OPENAI_API_KEY, opcionalno OPENAI_MODEL.
    Instaliraj: pip install openai
    """
    from openai import OpenAI

    client = OpenAI()
    model = os.getenv("OPENAI_MODEL", "gpt-4.1-mini")
    system = _build_system_prompt(task_ctx)

    resp = client.chat.completions.create(
        model=model,
        messages=[{"role": "system", "content": system}] + messages,
        temperature=0.2,
    )

```

```
return resp.choices[0].message.content
```

```
def call_llm(messages: List[Dict], task_ctx: Dict) -> str:
    use_local = os.getenv("USE_LOCAL", "1") == "1"
    if use_local:
        return call_ollama(messages, task_ctx)
    return call_openai(messages, task_ctx)
```

2) chat_ui.py

```
import streamlit as st
from typing import Dict
from llm_backends import call_llm

def _ensure_chat_state():
    if "messages" not in st.session_state:
        st.session_state.messages = [
            {"role": "assistant", "content": "Klikni 'Ne razumijem' ili napiši pitanje o zadatku."}
        ]

def reset_chat_if_task_changed(task_id: str):
    last = st.session_state.get("last_task_id")
    if last != task_id:
        st.session_state.last_task_id = task_id
        st.session_state.messages = [
            {"role": "assistant", "content": "Spreman. Reci što nije jasno oko ovog zadatka."}
        ]

def push_and_answer(user_text: str, task_ctx: Dict):
    _ensure_chat_state()
    st.session_state.messages.append({"role": "user", "content": user_text})
    answer = call_llm(st.session_state.messages, task_ctx)
    st.session_state.messages.append({"role": "assistant", "content": answer})

def render_chat(task_ctx: Dict):
    _ensure_chat_state()

    for m in st.session_state.messages:
        with st.chat_message(m["role"]):
            st.markdown(m["content"])

    user_prompt = st.chat_input("Pitaj nešto o zadatku...")
    if user_prompt:
        push_and_answer(user_prompt, task_ctx)
        st.rerun()
```

3) app.py (glavni Streamlit app)


```

import os
import pandas as pd
import streamlit as st
from chat_ui import render_chat, push_and_answer, reset_chat_if_task_changed

DATA_PATH = os.getenv("TASKS_CSV", "data/tasks.csv")

st.set_page_config(page_title="MI/ZI Tutor MVP", layout="wide")
st.title("MI/ZI Tutor MVP")

@st.cache_data
def load_tasks(csv_path: str) -> pd.DataFrame:
    df = pd.read_csv(csv_path)
    required = ["id", "exam_type", "task_text", "solution_text", "source_pdf", "page"]
    missing = [c for c in required if c not in df.columns]
    if missing:
        raise ValueError(f"tasks.csv nema kolone: {missing}")
    df["exam_type"] = df["exam_type"].astype(str).str.upper().str.strip()
    return df

if not os.path.exists(DATA_PATH):
    st.error(
        f"Ne nalazim CSV: {DATA_PATH}\n\n"
        "Napravi data/tasks.csv s kolonama: id, exam_type, task_text, solution_text, source_pdf, page"
    )
    st.stop()

df = load_tasks(DATA_PATH)

# Sidebar filteri
with st.sidebar:
    st.header("Filteri")

    mode = st.radio("Učim za:", ["MI", "ZI"], horizontal=True)
    query = st.text_input("Pretraži (po tekstu zadatka):", value="").strip()

    df_f = df[df["exam_type"] == mode].copy()
    if query:
        q = query.lower()
        df_f = df_f[df_f["task_text"].astype(str).str.lower().str.contains(q)]

    st.caption(f"Pronađeno zadataka: {len(df_f)}")

# Odabir zadatka
if df_f.empty:
    st.warning("Nema zadataka za ovaj filter.")
    st.stop()

# Napravi kratke nazive u listi (prvih 80 znakova)
df_f["label"] = df_f["id"].astype(str) + " | " + df_f["task_text"].astype(str).str.replace("\n", " ")

selected_label = st.selectbox("Odaberi zadatak:", df_f["label"].tolist(), index=0)

selected_row = df_f[df_f["label"] == selected_label].iloc[0]
task_ctx = {
    "id": str(selected_row["id"]),
    "exam_type": str(selected_row["exam_type"]),
    "task_text": str(selected_row["task_text"]),
    "solution_text": str(selected_row["solution_text"]),
    "source_pdf": str(selected_row["source_pdf"]),
    "page": int(selected_row["page"]),
}

reset_chat_if_task_changed(task_ctx["id"])

left, right = st.columns([1, 1])

with left:
    st.subheader(f"Zadatak ({task_ctx['exam_type']})")
    st.caption(f"Izvor: {task_ctx['source_pdf']}, str. {task_ctx['page']}")
    st.markdown(task_ctx["task_text"])

    with st.expander("Prikaži službeno rješenje"):
        st.markdown(task_ctx["solution_text"])

```

```
col1, col2 = st.columns([1, 1])
with col1:
    if st.button("Ne razumijem"):
        push_and_answer("Ne razumijem. Objasni rješenje korak po korak, jednostavno.", task_ctx)
        st.rerun()

with col2:
    if st.button("Daj šablonu"):
        push_and_answer(
            "Daj mi šablonu (checklistu) kako rješavati ovakve zadatke, na temelju ovog rješenja."
            task_ctx,
        )
        st.rerun()

with right:
    st.subheader("Chat")
    render_chat(task_ctx)
```

4) Pokretanje

Instaliraj:

```
pip install streamlit pandas requests
```

Lokalno (Ollama)

1. Instaliraj Ollama, pokreni:

```
ollama serve
ollama pull llama3.1
```

2. Pokreni app:

```
set USE_LOCAL=1
streamlit run app.py
```

Ako si na PowerShellu:

```
$env:USE_LOCAL="1"
streamlit run app.py
```

OpenAI

```
pip install openai
set USE_LOCAL=0
set OPENAI_API_KEY=...tvoj_kljuc
streamlit run app.py
```

Što je “stvarno” ovdje, i zašto nije hardkodirano

- Zadatke učitavaš iz `data/tasks.csv`.
- Filter “MI/ZI” radi na `exam_type`.
- “Ne razumijem” radi za bilo koji trenutno odabran zadatak, jer gumb samo šalje poruku i `task_ctx` u `call_lm`.
- Chat se resetira kad promijeniš zadatak, da se razgovori ne miješaju.

Ako mi zalijepiš primjer jednog retka iz tvog stvarnog indeksa (ili nazive kolona koje trenutno koristiš), mogu ti u istom stilu prilagoditi `load_tasks` da radi bez da mijenjaš export format.

```

asp index last3 mi last3 zi  Unable to display visualization
Vec imam csv, a moj app.py izgleda ovako: import streamlit as st
import pandas as pd
import fitz

st.set_page_config(page_title="ASP MVP", layout="wide")

df = pd.read_csv("data/asp_index_last3_mi_last3_zi.csv")

st.title("MVP za završni rad")

goal = st.selectbox("Cilj", ["prolaz (2)", "petica (5)"], key="goal")
days_left = st.slider("Dana do ispita", 0, 14, 1, key="days_left")
session_minutes = st.slider("Vrijeme za study blok (min/dan)", 30, 240, 120, key="session_minutes")

total_budget = session_minutes * max(1, days_left)

@st.cache_data
def render_pdf_page(pdf_path: str, page_1based: int, dpi: int = 150) -> bytes:
    doc = fitz.open(pdf_path)
    page_index = page_1based - 1
    if page_index < 0 or page_index >= len(doc):
        doc.close()
        raise ValueError("Stranica ne postoji u PDF-u.")
    page = doc[page_index]
    pix = page.get_pixmap(dpi=dpi)
    img = pix.tobytes("png")
    doc.close()
    return img

def get_weights():
    w_eff = 5.0
    w_points = 2.0
    w_freq = 0.5

    if goal == "prolaz (2)":
        w_detail = -3.0
        w_diff = -1.5
    else:
        w_detail = 1.0
        w_diff = -0.5

    if days_left <= 1:
        w_eff += 2.0
        w_points += 1.0
        w_detail -= 2.0

    return w_eff, w_points, w_detail, w_freq, w_diff

```

```
def score_breakdown(row):
    points = float(row["points"])
    time_est = float(row["time_est"])
    eff = points / time_est if time_est > 0 else 0.0
    is_detail = 1.0 if row["type"] == "detail" else 0.0
    freq = float(row.get("frequency_score", 0.0))
    diff = float(row.get("difficulty_guess", 0.0))

    w_eff, w_points, w_detail, w_freq, w_diff = get_weights()

    parts = {
        "efficiency": w_eff * eff,
        "points": w_points * points,
        "detail": w_detail * is_detail,
        "frequency": w_freq * freq,
        "difficulty": w_diff * diff
    }

    total = float(sum(parts.values()))
    return parts, total, {
        "eff": eff,
        "points": points,
        "time_est": time_est,
        "is_detail": is_detail,
        "freq": freq,
        "diff": diff,
        "weights": {
            "w_eff": w_eff,
            "w_points": w_points,
            "w_detail": w_detail,
            "w_freq": w_freq,
            "w_diff": w_diff
        }
    }

def score_task(row) -> float:
    _, total, _ = score_breakdown(row)
    return total

def build_plan(df_in: pd.DataFrame, minutes_budget: int) -> pd.DataFrame:
    df_local = df_in.copy()
    df_local["score"] = df_local.apply(score_task, axis=1)
    df_sorted = df_local.sort_values("score", ascending=False)

    selected_rows = []
    total_time = 0.0

    for _, row in df_sorted.iterrows():
        t = float(row["time_est"])
        if total_time + t <= float(minutes_budget):
```

```

        selected_rows.append(row)
        total_time += t

    return pd.DataFrame(selected_rows)

plan_df = build_plan(df, total_budget)

left, right = st.columns([1, 1])

with left:
    st.subheader("Preporuceni zadaci")

    if plan_df.empty:
        st.warning("Nema zadataka koji stanu u ovaj vremenski budzet. Povecaj minute ili provjeri time_est.")
        st.stop()

    st.caption(f"Ukupni budzet: {int(total_budget)} min (session_minutes * max(1, days_left))")

    total_points = float(plan_df["points"].sum())
    total_time = float(plan_df["time_est"].sum())
    avg_eff = (total_points / total_time) if total_time > 0 else 0.0

    m1, m2, m3 = st.columns(3)
    m1.metric("Ukupno bodova", f"{total_points:.0f}")
    m2.metric("Ukupno minuta", f"{total_time:.0f}")
    m3.metric("Prosjecni efficiency", f"{avg_eff:.3f}")

    st.dataframe(
        plan_df[["exam_type", "exam_date", "task_no", "points", "time_est", "type", "frequency_score", "difficulty_guess", "score"]],
        use_container_width=True
    )

with right:
    st.subheader("Pregled zadatka")

    task_labels = plan_df.apply(
        lambda r: f"{r['exam_type']} {r['exam_date']} - Zadatak {r['task_no']} (pts {r['points']}, {r['time_est']} min)",
        axis=1
    ).tolist()

    if "selected_task_label" not in st.session_state or st.session_state.selected_task_label not in task_labels:
        st.session_state.selected_task_label = task_labels[0]

    selected_task_label = st.selectbox(
        "Odaberi zadatak za pregled:",
        task_labels,
        key="selected_task_label"
    )

```

```

)

idx = task_labels.index(selected_task_label)
selected_task = plan_df.iloc[idx]

pdf_path = f"static/{selected_task['pdf']}"
page = int(selected_task["page"])

st.write(f"PDF: {selected_task['pdf']}, stranica {page}")

# Centered preview by using a narrower middle column
_, mid, _ = st.columns([1, 3, 1])
with mid:
    try:
        img_bytes = render_pdf_page(pdf_path, page)
        st.image(img_bytes, width=700)
    except Exception as e:
        st.error(f"Ne mogu otvoriti PDF stranicu: {e}")

st.subheader("Zasto je odabran ovaj zadatak")

parts, total_score, meta = score_breakdown(selected_task)

for key, value in parts.items():
    if value > 0:
        st.write(f"+ {key}: {value:.2f}")
    elif value < 0:
        st.write(f"- {key}: {value:.2f}")

st.markdown(f"***Ukupni score: {total_score:.2f}***")

with st.expander("Detalji (ulazi i tezine)"):
    st.write({
        "inputs": {
            "points": meta["points"],
            "time_est": meta["time_est"],
            "efficiency": meta["eff"],
            "type": "detail" if meta["is_detail"] == 1 else "core",
            "frequency_score": meta["freq"],
            "difficulty_guess": meta["diff"]
        },
        "weights": meta["weights"]
    })

```

Da, može. Tvoj CSV nema "solution_text", imaš samo **snippet** i referencu na PDF + stranicu. Znači "ne-hardkodirano" rješenje je:

- kad user odabere zadatak, ti već znaš **pdf** i **page**

- iz PDF-a izvučeš **tekst te stranice** (`fitz.get_text()`)
- taj tekst pošalješ modelu kao “službeni izvor” i kažeš mu da objašnjava samo iz toga
- UI: gumb “**Ne razumijem**” koji automatski pošalje poruku i okine odgovor

Ispod ti je **tvoj app.py**, ali proširen stvarnim chatbotom (radi s Ollama lokalno ili OpenAI). Nije hardkodirano, sve se veže na odabrani zadatak iz tvog `plan_df`.

app.py (copy paste)

```

import os
import requests
import streamlit as st
import pandas as pd
import fitz

st.set_page_config(page_title="ASP MVP", layout="wide")

# -----
# Config za LLM backend
# -----
# USE_LOCAL=1 -> Ollama (lokalno, bez tokena)
# USE_LOCAL=0 -> OpenAI (treba OPENAI_API_KEY)
USE_LOCAL = os.getenv("USE_LOCAL", "1") == "1"

OLLAMA_URL = os.getenv("OLLAMA_URL", "http://localhost:11434/api/chat")
OLLAMA_MODEL = os.getenv("OLLAMA_MODEL", "llama3.1")

OPENAI_MODEL = os.getenv("OPENAI_MODEL", "gpt-4.1-mini")

# -----
# Ucitavanje podataka
# -----
df = pd.read_csv("data/asp_index_last3_mi_last3_zi.csv")

st.title("MVP za završni rad")

goal = st.selectbox("Cilj", ["prolaz (2)", "petica (5)"], key="goal")
days_left = st.slider("Dana do ispita", 0, 14, 1, key="days_left")
session_minutes = st.slider("Vrijeme za study blok (min/dan)", 30, 240, 120, key="session_minutes")

total_budget = session_minutes * max(1, days_left)

# -----
# PDF helpers
# -----
@st.cache_data
def render_pdf_page(pdf_path: str, page_lbased: int, dpi: int = 150) -> bytes:
    doc = fitz.open(pdf_path)
    page_index = page_lbased - 1
    if page_index < 0 or page_index >= len(doc):
        doc.close()
        raise ValueError("Stranica ne postoji u PDF-u.")
    page = doc[page_index]
    pix = page.get_pixmap(dpi=dpi)
    img = pix.tobytes("png")
    doc.close()
    return img

@st.cache_data
def extract_pdf_page_text(pdf_path: str, page_lbased: int) -> str:
    doc = fitz.open(pdf_path)
    page_index = page_lbased - 1
    if page_index < 0 or page_index >= len(doc):
        doc.close()
        raise ValueError("Stranica ne postoji u PDF-u.")
    page = doc[page_index]
    text = page.get_text("text")
    doc.close()
    return (text or "").strip()

# -----
# Scoring
# -----
def get_weights():
    w_eff = 5.0
    w_points = 2.0
    w_freq = 0.5

    if goal == "prolaz (2)":
        w_detail = -3.0

```



```

        w_diff = -1.5
    else:
        w_detail = 1.0
        w_diff = -0.5

    if days_left <= 1:
        w_eff += 2.0
        w_points += 1.0
        w_detail -= 2.0

    return w_eff, w_points, w_detail, w_freq, w_diff


def score_breakdown(row):
    points = float(row['points'])
    time_est = float(row['time_est'])
    eff = points / time_est if time_est > 0 else 0.0
    is_detail = 1.0 if row['type'] == "detail" else 0.0
    freq = float(row.get("frequency_score", 0.0))
    diff = float(row.get("difficulty_guess", 0.0))

    w_eff, w_points, w_detail, w_freq, w_diff = get_weights()

    parts = {
        "efficiency": w_eff * eff,
        "points": w_points * points,
        "detail": w_detail * is_detail,
        "frequency": w_freq * freq,
        "difficulty": w_diff * diff
    }

    total = float(sum(parts.values()))
    return parts, total, {
        "eff": eff,
        "points": points,
        "time_est": time_est,
        "is_detail": is_detail,
        "freq": freq,
        "diff": diff,
        "weights": {
            "w_eff": w_eff,
            "w_points": w_points,
            "w_detail": w_detail,
            "w_freq": w_freq,
            "w_diff": w_diff
        }
    }

def score_task(row) -> float:
    _, total, _ = score_breakdown(row)
    return total

def build_plan(df_in: pd.DataFrame, minutes_budget: int) -> pd.DataFrame:
    df_local = df_in.copy()
    df_local["score"] = df_local.apply(score_task, axis=1)
    df_sorted = df_local.sort_values("score", ascending=False)

    selected_rows = []
    total_time = 0.0

    for _, row in df_sorted.iterrows():
        t = float(row["time_est"])
        if total_time + t <= float(minutes_budget):
            selected_rows.append(row)
            total_time += t

    return pd.DataFrame(selected_rows)

plan_df = build_plan(df, total_budget)

# -----
# . . . . .

```

```
# Chat state + LLM
# -----
def ensure_chat_state():
    if "chat_messages" not in st.session_state:
        st.session_state.chat_messages = [
            {"role": "assistant", "content": "Klikni 'Ne razumijem' ili napiši pitanje o trenutno odabranom zadatku."}
        ]

def reset_chat_if_task_changed(task_key: str):
    if st.session_state.get("last_task_key") != task_key:
        st.session_state.last_task_key = task_key
        st.session_state.chat_messages = [
            {"role": "assistant", "content": "Spreman. Pitaj što nije jasno oko ovog zadatka."}
        ]

def build_system_prompt(task_ctx: dict) -> str:
    return """
Ti si tutor koji objašnjava zadatke jednostavno, korak po korak.
SMIJEŠ koristiti samo informacije iz "IZVOR (PDF TEKST STRANICE)" i "SNIPPET".
Ne smiješ uvoditi alternativna rješenja ili drugačije korake od onih koji su u izvoru.
Ako u izvoru ne vidiš rješenje, jasno reci: "Na ovoj stranici ne vidim rješenje, pa ne mogu objasniti".

META:
- Predmet: {task_ctx.get("exam_type", "")}
- Datum: {task_ctx.get("exam_date", "")}
- Zadatak: {task_ctx.get("task_no", "")}
- PDF: {task_ctx.get("pdf", "")}
- Stranica: {task_ctx.get("page", "")}

SNIPPET (kratki opis iz indeksa):
{task_ctx.get("snippet", "")}

IZVOR (PDF TEKST STRANICE):
{task_ctx.get("page_text", "")}

FORMAT ODGOVORA:
1) Što se traži
2) Što je dano
3) Objašnjenje rješenja korak po korak (1..n)
4) Mini šablona: kako prepoznati ovakvu vrstu zadatka i što raditi
5) Brza provjera
"""

def call_ollama(messages: list[dict], task_ctx: dict) -> str:
    system = build_system_prompt(task_ctx)
    payload = {
        "model": OLLAMA_MODEL,
        "messages": [{"role": "system", "content": system}] + messages,
        "stream": False,
        "options": {"temperature": 0.2}
    }
    r = requests.post(OLLAMA_URL, json=payload, timeout=180)
    r.raise_for_status()
    data = r.json()
    return data["message"]["content"]

def call_openai(messages: list[dict], task_ctx: dict) -> str:
    from openai import OpenAI

    api_key = os.getenv("OPENAI_API_KEY", "").strip()
    if not api_key:
        return "Nema OPENAI_API_KEY u env var, a USE_LOCAL=0. Postavi ključ ili prebaci na lokalno (OLLAMA)."

    client = OpenAI(api_key=api_key)
    system = build_system_prompt(task_ctx)

    resp = client.chat.completions.create(
        model=OPENAI_MODEL,
        messages=[{"role": "system", "content": system}] + messages,
        temperature=0.2,
    )
    return resp.choices[0].message.content
```

```

return resp.choices[0].message.content

def call_llm(messages: list[dict], task_ctx: dict) -> str:
    if USE_LOCAL:
        return call_ollama(messages, task_ctx)
    return call_openai(messages, task_ctx)

def push_and_answer(user_text: str, task_ctx: dict):
    ensure_chat_state()
    st.session_state.chat_messages.append({'role': 'user', 'content': user_text})
    answer = call_llm(st.session_state.chat_messages, task_ctx)
    st.session_state.chat_messages.append({'role': 'assistant', 'content': answer})

# -----
# UI
# -----
left, right = st.columns([1, 1])

with left:
    st.subheader("Preporuceni zadaci")

    if plan_df.empty:
        st.warning('Nema zadataka koji stanu u ovaj vremenski budzet. Povecaj minute ili provjeri time')
        st.stop()

    st.caption(f'Ukupni budzet: {int(total_budget)} min (session_minutes * max(1, days_left))')

    total_points = float(plan_df['points'].sum())
    total_time = float(plan_df['time_est'].sum())
    avg_eff = (total_points / total_time) if total_time > 0 else 0.0

    m1, m2, m3 = st.columns(3)
    m1.metric("Ukupno bodova", f'{total_points:.0f}')
    m2.metric("Ukupno minuta", f'{total_time:.0f}')
    m3.metric("Prosjecni efficiency", f'{avg_eff:.3f}')

    st.dataframe(
        plan_df[['exam_type', 'exam_date', 'task_no', 'points', 'time_est', 'type',
                  'frequency_score', 'difficulty_guess', 'score']],
        use_container_width=True
    )

with right:
    st.subheader("Pregled zadatka")

    task_labels = plan_df.apply(
        lambda r: f'{r["exam_type"]} {r["exam_date"]} - Zadatak {r["task_no"]} (pts {r["points"]}, {r["time_est"]} min)',
        axis=1
    ).tolist()

    if "selected_task_label" not in st.session_state or st.session_state.selected_task_label not in task_labels:
        st.session_state.selected_task_label = task_labels[0]

    selected_task_label = st.selectbox(
        "Odaberi zadatak za pregled:",
        task_labels,
        key='selected_task_label'
    )

    idx = task_labels.index(selected_task_label)
    selected_task = plan_df.iloc[idx]

    pdf_path = f'static/{selected_task["pdf"]}'
    page = int(selected_task["page"])

    st.write(f'PDF: {selected_task["pdf"]}, stranica {page}')

    # Preview
    _, mid, _ = st.columns([1, 3, 1])
    with mid:
        try:
            img_bytes = render_pdf_page(pdf_path, page)
            st.image(img_bytes, width=700)

```

```

        st.image(img_bytes, width=700,
    except Exception as e:
        st.error(f"Ne mogu otvoriti PDF stranicu: {e}")

st.subheader("Zasto je odabran ovaj zadatak")

parts, total_score, meta = score_breakdown(selected_task)

for key, value in parts.items():
    if value > 0:
        st.write(f"+ {key}: {value:.2f}")
    elif value < 0:
        st.write(f"- {key}: {value:.2f}")

st.markdown(f"***Ukupni score: {total_score:.2f}***")

with st.expander("Detalji (ulazi i tezine)"):
    st.write({
        "inputs": {
            "points": meta["points"],
            "time_est": meta["time_est"],
            "efficiency": meta["eff"],
            "type": "detail" if meta["is_detail"] == 1 else "core",
            "frequency_score": meta["freq"],
            "difficulty_guess": meta["diff"]
        },
        "weights": meta["weights"]
    })

# -----
# Chatbot za trenutno odabrani zadatak
# -----
st.divider()
st.subheader("Chat: objasnjenje rjesenja")

# Task context iz odabranog reda + tekst stranice
task_key = f"{selected_task['exam_type']}|{selected_task['exam_date']}|{selected_task['task_no']}"
reset_chat_if_task_changed(task_key)

try:
    page_text = extract_pdf_page_text(pdf_path, page)
except Exception as e:
    page_text = ""
    st.warning(f"Ne mogu izvuci tekst sa stranice za chat: {e}")

task_ctx = {
    "exam_type": str(selected_task["exam_type"]),
    "exam_date": str(selected_task["exam_date"]),
    "task_no": int(selected_task["task_no"]),
    "pdf": str(selected_task["pdf"]),
    "page": int(page),
    "snippet": str(selected_task.get("snippet", "")),
    "page_text": page_text
}

colA, colB = st.columns([1, 1])
with colA:
    if st.button("Ne razumijem"):
        push_and_answer("Ne razumijem zadatak. Objasni rjesenje jednostavno, korak po korak.", ta
        st.rerun()
with colB:
    if st.button("Daj sablonu"):
        push_and_answer("Daj mi sablon (checklistu) kako rjesavati ovakve zadatke, na temelju ovog
        st.rerun()

ensure_chat_state()
for m in st.session_state.chat_messages:
    with st.chat_message(m["role"]):
        st.markdown(m["content"])

user_prompt = st.chat_input("Pitaj nesto o ovom zadatku...")
if user_prompt:
    push_and_answer(user_prompt, task_ctx)
    st.rerun()

```

Kako to pokrenuti stvarno

Opcija 1: Lokalno (Ollama, bez tokena)

1. Pokreni Ollama server i preuzmi model:

- `ollama serve`
- `ollama pull llama3.1`

2. Env var:

- `USE_LOCAL=1`

3. Run:

- `streamlit run app.py`

Ako ti Ollama nije na default portu, postavi `OLLAMA_URL`.

Opcija 2: OpenAI

1. `pip install openai`

2. Env var:

- `USE_LOCAL=0`
- `OPENAI_API_KEY=...`

Bitna napomena

Ovaj bot objašnjava “službeno” samo ako je na toj PDF stranici stvarno i rješenje, ne samo tekst zadatka. Ako su rješenja na drugim stranicama, najbolji upgrade je: u CSV dodati i `solution_page` ili `solution_pdf` (može i isti pdf), pa bot čita baš to.

Ako mi kažeš jesu li rješenja na istoj stranici ili drugdje, mogu ti odmah prilagoditi da za chat vuče i “task page” i “solution page” i spaja ih u kontekst.

mozemo li zasad samo napraviti da llm odgovara sa rjesenjem koje ne koristi nuzno rjesenje iz ikakvog sluzbenog solutiona, nego da samo odgovara s malo prilagodenim promptom bez da mu user mora rucno tipkati.

Možemo, i to je još lakše: “Ne razumijem” gumb će samo poslati LLM-u tekst zadatka (iz PDF stranice ili iz `snippet`) i reći mu da riješi i objasni, bez ikakvog “službenog rješenja”.

U tvom trenutnom appu trebaš samo 3 stvari:

1. izvući tekst stranice (to već imamo funkciju `extract_pdf_page_text`)

2. promijeniti prompt da je "Solve + explain" umjesto "Explain official solution"
3. "Ne razumijem" gumb da automatski pošalje tu poruku

Ispod su minimalne izmjene koje možeš copy pasteati u svoj postojeći `app.py`.

1) Zamijeni prompt builder

U tvom kodu gdje si imao `build_system_prompt(task_ctx)`, zamijeni ga ovim:

```
def build_system_prompt(task_ctx: dict) -> str:
    return """
Ti si tutor za ASP zadatke. Tvoj posao je riješiti zadatak i objasniti rješenje jednostavno, korak po korak.
Ne trebaš imati službeno rješenje. Ako nešto nedostaje u tekstu zadatka, postavi razumnu pretpostavku.

META:
- Predmet: {task_ctx.get("exam_type", "")}
- Datum: {task_ctx.get("exam_date", "")}
- Zadatak: {task_ctx.get("task_no", "")}
- PDF: {task_ctx.get("pdf", "")}
- Stranica: {task_ctx.get("page", "")}

TEKST ZADATKA (iz PDF stranice):
{task_ctx.get("page_text", "")}

DODATNI KONTEKST (snippet iz indeksa, može pomoći):
{task_ctx.get("snippet", "")}

FORMAT ODGOVORA:
1) Što se traži
2) Što je dano (i pretpostavke, ako treba)
3) Rješenje, korak po korak
4) Finalni odgovor (jasno označi)
5) Ako je relevantno: provjera ili intuition check
""".strip()
```

Ovim si maknuo bilo kakvu ovisnost o "službenom rješenju".

2) Napravi "Ne razumijem" da šalje automatski solve prompt

U tvom UI dijelu gdje je chat (na desnoj strani, ispod previewa), zamijeni gumb handler s ovim:

```
if st.button("Ne razumijem"):
    push_and_answer(
        "Riješi ovaj zadatak i objasni rješenje jednostavno, korak po korak.",
        task_ctx
    )
    st.rerun()
```

To je to. User ne tipka ništa.

Ako želiš i drugi gumb "Samo rješenje":

```
if st.button("Samo rješenje"):
    push_and_answer(
        "Riješi zadatak i daj samo finalni odgovor, bez objašnjenja.",
        task_ctx
    )
    st.rerun()
```

3) Bitno: pobrini se da `task_ctx["page_text"]` nije prazan

Ti već imaš:

```
page_text = extract_pdf_page_text(pdf_path, page)
```

Ako ti nekad vrati prazan tekst (PDF je sken), LLM neće imati što rješavati. Za MVP, u tom slučaju možeš fallbackati na `snippet`:

```
if not page_text.strip():
    page_text = str(selected_task.get("snippet", "")).strip()
```

To ubaci odmah nakon `extract_pdf_page_text(...)`.

Bonus: da ne bude “previše generički” (malo bolji solve prompt)

Ako želiš da odgovori više kao “ispitno rješenje”, a ne kao chat, promijeni user tekst na:

```
push_and_answer(
    "Rijesi zadatak kao na ispitu. Koristi jasne korake, formule i zaključak. Ne preskači ključne korake."
    task_ctx
)
```

Što dobivaš ovim

- Radi odmah, bez dodavanja novih kolona u CSV.
- UI ostaje ultra jednostavan.
- Chat je kontekstualan jer uvijek dobije baš tekst trenutno odabrane PDF stranice.

Ako želiš, mogu ti dati i varijantu gdje LLM automatski prvo iz “page_text” pokuša izolirati samo dio koji izgleda kao tekst zadatka (npr. do “Rješenje” ili do kraja prvog bloka), da ne žvače cijelu stranicu kad na njoj ima više stvari.

super, posalji mi sad puni kod svakog filea da ih mogu samo copy pasteati.

Naravno. Ispod su 3 filea koja možeš staviti u isti folder (uz tvoj postojeći `data/...csv` i `static/...pdf` folder). Ovo je verzija gdje LLM rješava zadatak iz teksta PDF stranice, bez “službenog rješenja”. “Ne razumijem” je jedan klik, user ne tipka ništa.

Nema hardkodiranih zadataka, sve dolazi iz tvog CSV-a i trenutno odabranog reda u planu.

1) llm_backends.py

```
import os
import requests
from typing import List, Dict

USE_LOCAL = os.getenv("USE_LOCAL", "1") == "1"

OLLAMA_URL = os.getenv("OLLAMA_URL", "http://localhost:11434/api/chat")
OLLAMA_MODEL = os.getenv("OLLAMA_MODEL", "llama3.1")
OLLAMA_TEMPERATURE = float(os.getenv("OLLAMA_TEMPERATURE", "0.2"))

OPENAI_MODEL = os.getenv("OPENAI_MODEL", "gpt-4.1-mini")

def build_system_prompt(task_ctx: Dict) -> str:
    return """
    Ti si tutor za ASP zadatke. Tvoj posao je riješiti zadatak i objasniti rješenje jednostavno, korak po korak.
    Ne trebaš imati službeno rješenje. Ako nešto nedostaje u tekstu zadatka, postavi razumnu pretpostavku.
    Piši na hrvatskom.

    META:
    - Predmet: {task_ctx.get("exam_type", "")}
    - Datum: {task_ctx.get("exam_date", "")}
    - Zadatak: {task_ctx.get("task_no", "")}
    - PDF: {task_ctx.get("pdf", "")}
    - Stranica: {task_ctx.get("page", "")}

    TEKST ZADATKA (iz PDF stranice):
    {task_ctx.get("page_text", "")}

    DODATNI KONTEKST (snippet iz indeksa, može pomoći):
    {task_ctx.get("snippet", "")}

    FORMAT ODGOVORA:
    1) Što se traži
    2) Što je dano (i pretpostavke, ako treba)
    3) Rješenje, korak po korak
    4) Finalni odgovor (jasno označi)
    5) Ako je relevantno: provjera ili intuition check
    """

def call_ollama(messages: List[Dict], task_ctx: Dict) -> str:
    system = build_system_prompt(task_ctx)
    payload = {
        "model": OLLAMA_MODEL,
        "messages": [{"role": "system", "content": system}] + messages,
        "stream": False,
        "options": {
            "temperature": OLLAMA_TEMPERATURE
        }
    }
    r = requests.post(OLLAMA_URL, json=payload, timeout=180)
    r.raise_for_status()
    data = r.json()
    return data["message"]["content"]

def call_openai(messages: List[Dict], task_ctx: Dict) -> str:
    api_key = os.getenv("OPENAI_API_KEY", "").strip()
    if not api_key:
        return "Nema OPENAI_API_KEY u env var, a USE_LOCAL=0. Postavi ključ ili prebaci na lokalno (U: "

    from openai import OpenAI
    client = OpenAI(api_key=api_key)

    system = build_system_prompt(task_ctx)

    resp = client.chat.completions.create(
        model=OPENAI_MODEL,
        messages=[{"role": "system", "content": system}] + messages,
        temperature=0.2,
    )
    return resp.choices[0].message.content
```



```
def call_llm(messages: List[Dict], task_ctx: Dict) -> str:
    if USE_LOCAL:
        return call_ollama(messages, task_ctx)
    return call_openai(messages, task_ctx)
```

2) chat_ui.py

```
import streamlit as st
from typing import Dict
from llm_backends import call_llm

def ensure_chat_state():
    if "chat_messages" not in st.session_state:
        st.session_state.chat_messages = [
            {"role": "assistant", "content": "Klikni 'Ne razumijem' ili napiši pitanje o trenutno odabranom zadatku."}
        ]

def reset_chat_if_task_changed(task_key: str):
    if st.session_state.get("last_task_key") != task_key:
        st.session_state.last_task_key = task_key
        st.session_state.chat_messages = [
            {"role": "assistant", "content": "Spreman. Pitaj što nije jasno oko ovog zadatka."}
        ]

def push_and_answer(user_text: str, task_ctx: Dict):
    ensure_chat_state()
    st.session_state.chat_messages.append({"role": "user", "content": user_text})
    answer = call_llm(st.session_state.chat_messages, task_ctx)
    st.session_state.chat_messages.append({"role": "assistant", "content": answer})

def render_chat(task_ctx: Dict):
    ensure_chat_state()

    for m in st.session_state.chat_messages:
        with st.chat_message(m["role"]):
            st.markdown(m["content"])

    user_prompt = st.chat_input("Pitaj nešto o ovom zadatku...")
    if user_prompt:
        push_and_answer(user_prompt, task_ctx)
        st.rerun()
```

3) app.py

Ovo je tvoj postojeći `app.py` + integriran chat (bez hardkodiranja).

```

import streamlit as st
import pandas as pd
import fitz

from chat_ui import render_chat, push_and_answer, reset_chat_if_task_changed

st.set_page_config(page_title="ASP MVP", layout="wide")

df = pd.read_csv("data/asp_index_last3_mi_last3_zi.csv")

st.title('MVP za završni rad')

goal = st.selectbox("Cilj", ["prolaz (2)", "petica (5)"], key="goal")
days_left = st.slider("Dana do ispita", 0, 14, 1, key="days_left")
session_minutes = st.slider("Vrijeme za study blok (min/dan)", 30, 240, 120, key="session_minutes")

total_budget = session_minutes * max(1, days_left)

@st.cache_data
def render_pdf_page(pdf_path: str, page_lbased: int, dpi: int = 150) -> bytes:
    doc = fitz.open(pdf_path)
    page_index = page_lbased - 1
    if page_index < 0 or page_index >= len(doc):
        doc.close()
        raise ValueError("Stranica ne postoji u PDF-u.")
    page = doc[page_index]
    pix = page.get_pixmap(dpi=dpi)
    img = pix.tobytes("png")
    doc.close()
    return img

@st.cache_data
def extract_pdf_page_text(pdf_path: str, page_lbased: int) -> str:
    doc = fitz.open(pdf_path)
    page_index = page_lbased - 1
    if page_index < 0 or page_index >= len(doc):
        doc.close()
        raise ValueError("Stranica ne postoji u PDF-u.")
    page = doc[page_index]
    text = page.get_text("text")
    doc.close()
    return (text or "").strip()

def get_weights():
    w_eff = 5.0
    w_points = 2.0
    w_freq = 0.5

    if goal == "prolaz (2)":
        w_detail = -3.0
        w_diff = -1.5
    else:
        w_detail = 1.0
        w_diff = -0.5

    if days_left <= 1:
        w_eff += 2.0
        w_points += 1.0
        w_detail -= 2.0

    return w_eff, w_points, w_detail, w_freq, w_diff

def score_breakdown(row):
    points = float(row['points'])
    time_est = float(row['time_est'])
    eff = points / time_est if time_est > 0 else 0.0
    is_detail = 1.0 if row['type'] == "detail" else 0.0
    freq = float(row.get("frequency_score", 0.0))
    diff = float(row.get("difficulty_guess", 0.0))

    w_eff, w_points, w_detail, w_freq, w_diff = get_weights()

```

```

parts = {
    "efficiency": w_eff * eff,
    "points": w_points * points,
    "detail": w_detail * is_detail,
    "frequency": w_freq * freq,
    "difficulty": w_diff * diff
}

total = float(sum(parts.values()))
return parts, total, {
    "eff": eff,
    "points": points,
    "time_est": time_est,
    "is_detail": is_detail,
    "freq": freq,
    "diff": diff,
    "weights": {
        "w_eff": w_eff,
        "w_points": w_points,
        "w_detail": w_detail,
        "w_freq": w_freq,
        "w_diff": w_diff
    }
}

def score_task(row) -> float:
    _, total, _ = score_breakdown(row)
    return total

def build_plan(df_in: pd.DataFrame, minutes_budget: int) -> pd.DataFrame:
    df_local = df_in.copy()
    df_local["score"] = df_local.apply(score_task, axis=1)
    df_sorted = df_local.sort_values("score", ascending=False)

    selected_rows = []
    total_time = 0.0

    for _, row in df_sorted.iterrows():
        t = float(row["time_est"])
        if total_time + t <= float(minutes_budget):
            selected_rows.append(row)
            total_time += t

    return pd.DataFrame(selected_rows)

plan_df = build_plan(df, total_budget)

left, right = st.columns([1, 1])

with left:
    st.subheader("Preporuceni zadaci")

    if plan_df.empty:
        st.warning("Nema zadataka koji stanu u ovaj vremenski budzet. Povecaj minute ili provjeri tim")
        st.stop()

    st.caption(f"Ukupni budzet: {int(total_budget)} min (session_minutes * max(1, days_left))")

    total_points = float(plan_df["points"].sum())
    total_time = float(plan_df["time_est"].sum())
    avg_eff = (total_points / total_time) if total_time > 0 else 0.0

    m1, m2, m3 = st.columns(3)
    m1.metric("Ukupno bodova", f"{total_points:.0f}")
    m2.metric("Ukupno minuta", f"{total_time:.0f}")
    m3.metric("Prosjecni efficiency", f"{avg_eff:.3f}")

    st.dataframe(
        plan_df[["exam_type", "exam_date", "task_no", "points", "time_est", "type", "frequency_score",
        use_container_width=True
    )

```

```

with right:
    st.subheader('Pregled zadatka')

    task_labels = plan_df.apply(
        lambda r: f'{r["exam_type"]} {r["exam_date"]} - Zadatak {r["task_no"]} (pts {r["points"]}, {r["time_est"]})',
        axis=1
    ).tolist()

    if "selected_task_label" not in st.session_state or st.session_state.selected_task_label not in task_labels:
        st.session_state.selected_task_label = task_labels[0]

    selected_task_label = st.selectbox(
        "Odaberi zadatak za pregled:",
        task_labels,
        key="selected_task_label"
    )

    idx = task_labels.index(selected_task_label)
    selected_task = plan_df.iloc[idx]

    pdf_path = f"static/{selected_task['pdf']}"
    page = int(selected_task["page"])

    st.write(f"PDF: {selected_task['pdf']}, stranica {page}")

    # Centered preview by using a narrower middle column
    _, mid, _ = st.columns([1, 3, 1])
    with mid:
        try:
            img_bytes = render_pdf_page(pdf_path, page)
            st.image(img_bytes, width=700)
        except Exception as e:
            st.error(f"Ne mogu otvoriti PDF stranicu: {e}")

    st.subheader("Zasto je odabran ovaj zadatak")

    parts, total_score, meta = score_breakdown(selected_task)

    for key, value in parts.items():
        if value > 0:
            st.write(f"+ {key}: {value:.2f}")
        elif value < 0:
            st.write(f"- {key}: {value:.2f}")

    st.markdown(f"**Ukupni score: {total_score:.2f}")

    with st.expander("Detalji (ulazi i tezine)"):
        st.write({
            "inputs": {
                "points": meta["points"],
                "time_est": meta["time_est"],
                "efficiency": meta["eff"],
                "type": "detail" if meta["is_detail"] == 1 else "core",
                "frequency_score": meta["freq"],
                "difficulty_guess": meta["diff"]
            },
            "weights": meta["weights"]
        })

    # -----
    # Chat (solve + explain)
    # -----
    st.divider()
    st.subheader("Chat: rjesenje i objasnjenje")

    task_key = f'{selected_task["exam_type"]}|{selected_task["exam_date"]}|{selected_task["task_no"]}'
    reset_chat_if_task_changed(task_key)

    try:
        page_text = extract_pdf_page_text(pdf_path, page)
    except Exception as e:
        page_text = ""
        st.warning(f"Ne mogu izvuci tekst sa stranice za chat: {e}")

    # Feedback ako je PDF stranica i ona takoda

```

```
# fallback ako je PDF skeniran i nema teksta
if not page_text.strip():
    page_text = str(selected_task.get("snippet", "")).strip()

task_ctx = {
    "exam_type": str(selected_task["exam_type"]),
    "exam_date": str(selected_task["exam_date"]),
    "task_no": int(selected_task["task_no"]),
    "pdf": str(selected_task["pdf"]),
    "page": int(page),
    "snippet": str(selected_task.get("snippet", "")),
    "page_text": page_text
}

colA, colB = st.columns([1, 1])
with colA:
    if st.button("Ne razumijem"):
        push_and_answer(
            "Rijesi ovaj zadatak i objasni rješenje jednostavno, korak po korak.",
            task_ctx
        )
        st.rerun()

with colB:
    if st.button("Samo finalni odgovor"):
        push_and_answer(
            "Rijesi zadatak i vrati samo finalni odgovor. Bez objašnjenja.",
            task_ctx
        )
        st.rerun()

render_chat(task_ctx)
```

Pokretanje

Lokalno (Ollama)

1. instaliraj Ollama
2. u terminalu:

```
ollama serve
ollama pull llama3.1
```

3. pokreni Streamlit:

```
pip install streamlit pandas pymupdf requests
set USE_LOCAL=1
streamlit run app.py
```

OpenAI

```
pip install openai
set USE_LOCAL=0
set OPENAI_API_KEY=...tvoj_kljuc
streamlit run app.py
```

Ako ti javiš da su neki PDF-ovi skenirani pa `page.get_text()` vrati prazno često, najbolji idući upgrade je da u CSV dodaš čist `task_text` (iz indeksa ili ručno za top N), pa ne ovisiš o ekstrakciji iz PDF-a.

moгу li imati svoj api key ako imam chat gpt pro? moram li ista placati dodatno?

Kratko i jasno:

Ne. ChatGPT Pro ne daje ti automatski API key.

I da, API se plaća odvojeno.

To su dvije potpuno odvojene stvari:

1 ChatGPT Plus / Pro (ono što ti imaš)

To je pretplata za korištenje ChatGPT-a u web aplikaciji.

- Plaćaš mjesečnu pretplatu.
- Možeš koristiti modele unutar chat.openai.com.
- Nemaš API ključ uključen u to.
- Ne možeš taj plan "spojiti" direktno na svoj Streamlit app.

To je kao Netflix za ChatGPT UI.

2 OpenAI API (za tvoju aplikaciju)

To je zasebna usluga.

- Trebaš otvoriti account na platform.openai.com
- Generiraš API key
- Dodaš billing (karticu)
- Plaćaš po potrošnji tokena

To je pay-as-you-go, nema mjesečne pretplate.

Koliko bi te to realno koštalo za tvoj projekt?

Za MVP gdje:

- rješavaš tekst jednog zadatka
- odgovori su možda 500–1500 tokena
- koristiš nešto poput `gpt-4.1-mini`

To su doslovno **centi po odgovoru**.

Ako koristiš lokalni model (Ollama), onda je:

- 0 € po pozivu
- ali kvaliteta nešto slabija
- treba ti malo jači RAM

Što ti preporučujem

Za tvoj završni rad:

♦ Faza 1

Koristi lokalni model (USE_LOCAL=1)

Besplatno, dovoljno dobro za demo i development.

♦ Faza 2 (kad želiš wow efekt)

Prebaci na OpenAI API za:

- bolju matematičku preciznost
- stabilnije objašnjenje
- manje halucinacija

To uključiš samo kad trebaš pokazati komisiji.

Važna stvar koju ljudi često brkaju

ChatGPT Pro ≠ API plan

To su dva potpuno odvojena billing sustava.

Ako želiš, mogu ti:

- procijeniti mjesečni trošak za 1000 korisnika
- ili ti pokazati kako optimizirati potrošnju tokena u tvom kodu da bude minimalna
- ili usporediti koji lokalni model ti je best trade-off za ASP zadatke

Što te više zanima?

```
tried ollama serve, got: Error: listen tcp 127.0.0.1:11434: bind: Only one usage of each socket address (protocol/network address/port) is normally permitted.
```

Odlično, ovo je čista tehnička stvar.

Ta poruka znači:

Port 11434 je već zauzet.

Ollama server već radi ili ga nešto drugo koristi.

Nije greška modela, nego mrežni konflikt.

✓ Najčešći scenarij (95% slučajeva)

Ollama već radi u pozadini.

Probaj samo:

```
ollama list
```

Ako dobiješ listu modela, server je već aktivan i **ne trebaš pokretati ollama serve ponovno**.

U tom slučaju samo pokreni Streamlit:

```
streamlit run app.py
```

🔍 Ako nisi siguran radi li Ollama

U browser upiši:

```
http://localhost:11434
```

Ako dobiješ odgovor poput:

```
Ollama is running
```

Sve je OK.

🔧 Ako želiš vidjeti što koristi port

U Windows Command Promptu:

```
netstat -ano | findstr 11434
```

Vidjet ćeš PID procesa.

Zatim:

```
tasklist | findstr <PID>
```

Ali opet, najčešće je to već Ollama.

🚫 Ako želiš ubiti postojeći Ollama proces

Samo ako stvarno treba:

```
taskkill /PID <PID> /F
```

Pa onda opet:

```
ollama serve
```

💡 Najvažnije

Ne trebaš ručno pokretati `ollama serve` svaki put.

Ollama se često sam pokrene kao servis.

Brzi test da vidiš radi li model

U terminal:

```
ollama run llama3.1
```

Ako ti otvori interaktivni chat, sve radi.

Ako želiš, mogu ti odmah dati:

- ultra lagan model koji je brži za ASP (npr. manji od llama3.1)
- ili optimalan model za matematičke zadatke koji je precizniji

Reci kakav ti je RAM i CPU.

runnao sam ollamu. sto sad? kako da napravim da radi u aplikaciji?

Super. Ako `ollama run ...` radi, to znači da je lokalni server spreman. Sad samo trebaš 3 stvari da bi Streamlit app komunicirao s njim.

1) Provjeri da Ollama API stvarno sluša na 11434

U terminalu upiši:

```
curl http://localhost:11434/api/tags
```

Ako dobiješ JSON s modelima, to je to.

Ako nemaš `curl` na Windowsu, može i u PowerShellu:

```
Invoke-RestMethod http://localhost:11434/api/tags
```

2) Postavi env var za aplikaciju

U folderu gdje ti je `app.py`:

PowerShell

```
$env:USE_LOCAL="1"
$env:OLLAMA_URL="http://localhost:11434/api/chat"
$env:OLLAMA_MODEL="llama3.1"
streamlit run app.py
```

CMD

```
set USE_LOCAL=1
set OLLAMA_URL=http://localhost:11434/api/chat
set OLLAMA_MODEL=llama3.1
streamlit run app.py
```

To je dovoljno ako koristiš moj `llm_backends.py` koji zove Ollama preko HTTP-a.

3) Provjeri da ti je instaliran requests

U istom venvu gdje pokrećeš Streamlit:

```
pip install requests
```

4) Brzi “sanity check” iz Pythona (ako app ne odgovara)

Napravimo mini test script `test_ollama.py` u istom folderu:

```
import requests

payload = {
    "model": "llama3.1",
    "messages": [{ "role": "user", "content": "Reci samo: OK" }],
    "stream": False
}

r = requests.post('http://localhost:11434/api/chat', json=payload, timeout=60)
r.raise_for_status()
print(r.json()[ "message" ][ 'content' ])
```

Pokreni:

```
python test_ollama.py
```

Ako ispiše “OK”, 100% je povezivo, i problem je onda u tvom Streamlit kodu ili env varovima.

5) Što očekuješ u aplikaciji kad radi

U appu:

- odabereš zadatak
- klikneš “Ne razumijem”
- trebala bi se pojaviti poruka usera + odgovor bota u chat UI-u

Ako se ne pojavljuje:

- provjeri da ti se `llm_backends.py` i `chat_ui.py` nalaze uz `app.py`
- provjeri da u `llm_backends.py` model ime točno odgovara (`llama3.1` vs `llama3` itd.)
- provjeri Streamlit log u terminalu, obično izbací `ConnectionError` ako URL nije dobar

Najčešći failovi i fix

- **Model nije povučen:** `ollama pull llama3.1`
- **Krivi model string:** u terminalu `ollama list` pa kopiraj točno ime
- **Krivi URL:** mora biti `http://localhost:11434/api/chat`
- **Ollama nije server, nego samo run:** svejedno server radi u pozadini, ali ako ne, otvori Ollama app (Windows) ili restartaj Ollama

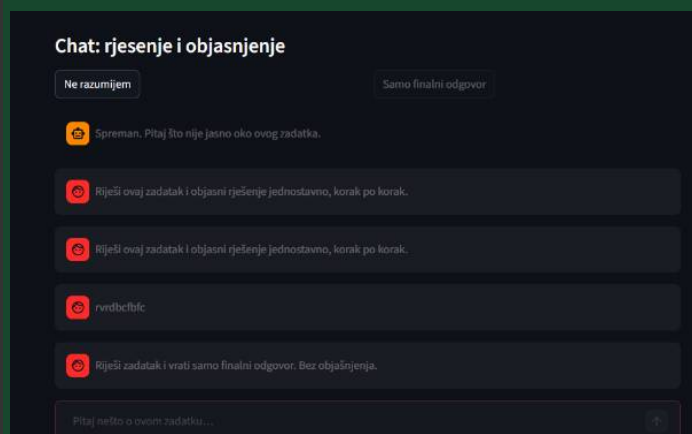
Ako mi zalijepiš:

1. output od `ollama list`

2. output od `Invoke-RestMethod http://localhost:11434/api/tags` (prva 3 reda)

3. screenshot ili copy paste errora iz Streamlit terminala

reći ću ti točno gdje puca i što promijeniti.



ovako izgleda output trenutacno. ne cini mi se kao da radi.

Vidim što se događa: u chatu ti se dodaju user poruke, ali **assistant poruka se uopće ne pojavi**. To gotovo uvijek znači da `push_and_answer()` pukne usred poziva (exception) pa nikad ne dođe do `append` assistant odgovora.

Najbrže ćemo to riješiti u 2 koraka:

1. **napraviti da se greška prikaže u UI-u** (ne samo u terminalu)
2. **popraviti Ollama poziv tako da radi i ako tvoja Ollama verzija ne podržava `/api/chat`** (neke instalacije rade samo `/api/generate`)

Ispod ti dajem copy paste izmjene.

Korak 1: U `chat_ui.py` uhvati grešku i ispiši je kao assistant poruku

Zamijeni cijeli `chat_ui.py` ovim:

```
import streamlit as st
from typing import Dict
from llama_backends import call_llm

def ensure_chat_state():
    if "chat_messages" not in st.session_state:
        st.session_state.chat_messages = [
            {"role": "assistant", "content": "Klikni 'Ne razumijem' ili napiši pitanje o trenutno odabranom zadatku."}
        ]

def reset_chat_if_task_changed(task_key: str):
    if st.session_state.get("last_task_key") != task_key:
        st.session_state.last_task_key = task_key
        st.session_state.chat_messages = [
            {"role": "assistant", "content": "Spreman. Pitaj što nije jasno oko ovog zadatka."}
        ]

def push_and_answer(user_text: str, task_ctx: Dict):
    ensure_chat_state()
    st.session_state.chat_messages.append({"role": "user", "content": user_text})

    try:
        answer = call_llm(st.session_state.chat_messages, task_ctx)
        if not answer or not str(answer).strip():
            answer = "Dobio sam prazan odgovor od modela. Provjeri Ollama endpoint i model name."
        st.session_state.chat_messages.append({"role": "assistant", "content": answer})
    except Exception as e:
        st.session_state.chat_messages.append(
            {"role": "assistant", "content": f"Greška pri pozivu modela: {type(e).__name__}: {e}"}
        )

def render_chat(task_ctx: Dict):
    ensure_chat_state()

    for m in st.session_state.chat_messages:
        with st.chat_message(m["role"]):
            st.markdown(m["content"])

    user_prompt = st.chat_input("Pitaj nešto o ovom zadatku...")
    if user_prompt:
        push_and_answer(user_prompt, task_ctx)
        st.rerun()
```

Ovo će ti odmah u UI-u pokazati je li to `ConnectionError`, `HTTPError`, `KeyError`, ili nešto četvrto.

Korak 2: Napravi `llm_backends.py` kompatibilan s Ollama `/api/chat` i `/api/generate`

Zamijeni cijeli `llm_backends.py` ovim:

```
import os
import requests
from typing import List, Dict

USE_LOCAL = os.getenv("USE_LOCAL", "1") == "1"

OLLAMA_BASE = os.getenv("OLLAMA_BASE", "http://localhost:11434")
OLLAMA_MODEL = os.getenv("OLLAMA_MODEL", "llama3.1")
OLLAMA_TEMPERATURE = float(os.getenv("OLLAMA_TEMPERATURE", "0.2"))

OPENAI_MODEL = os.getenv("OPENAI_MODEL", "gpt-4.1-mini")

def build_system_prompt(task_ctx: Dict) -> str:
    return f"""
    Ti si tutor za ASP zadatke. Riješi zadatak i objasni rješenje jednostavno, korak po korak.
    Ako nešto nedostaje, postavi razumnu pretpostavku i jasno napiši pretpostavku.
    Piši na hrvatskom.

    META:
    - Predmet: {task_ctx.get("exam_type", "")}
    - Datum: {task_ctx.get("exam_date", "")}
    - Zadatak: {task_ctx.get("task_no", "")}
    - PDF: {task_ctx.get("pdf", "")}
    - Stranica: {task_ctx.get("page", "")}

    TEKST ZADATKA:
    {task_ctx.get("page_text", "")}

    DODATNI KONTEKST (snippet):
    {task_ctx.get("snippet", "")}

    FORMAT:
    1) Što se traži
    2) Što je dano (i pretpostavke)
    3) Rješenje korak po korak
    4) Finalni odgovor
    """.strip()

def _ollama_chat(messages: List[Dict], system: str) -> str:
    url = f"{OLLAMA_BASE}/api/chat"
    payload = {
        "model": OLLAMA_MODEL,
        "messages": [{"role": "system", "content": system}] + messages,
        "stream": False,
        "options": {"temperature": OLLAMA_TEMPERATURE},
    }
    r = requests.post(url, json=payload, timeout=180)
    r.raise_for_status()
    data = r.json()
    return data["message"]["content"]

def _ollama_generate(messages: List[Dict], system: str) -> str:
    """
    Fallback za starije Ollama verzije ili kad /api/chat ne radi.
    /api/generate očekuje jedan prompt string.
    """
    url = f"{OLLAMA_BASE}/api/generate"

    convo = []
    for m in messages[-6:]:
        role = m.get("role", "user")
        content = m.get("content", "")
        convo.append(f"{role.upper()}: {content}")

    prompt = system + "\n\n" + "\n".join(convo) + "\nASSISTANT:"

    payload = {
        "model": OLLAMA_MODEL,
        "prompt": prompt,
        "stream": False,
        "options": {"temperature": OLLAMA_TEMPERATURE},
    }
```

```

r = requests.post(url, json=payload, timeout=180)
r.raise_for_status()
data = r.json()
return data.get("response", "").strip()

def call_ollama(messages: List[Dict], task_ctx: Dict) -> str:
    system = build_system_prompt(task_ctx)

    try:
        return _ollama_chat(messages, system)
    except Exception:
        return _ollama_generate(messages, system)

def call_openai(messages: List[Dict], task_ctx: Dict) -> str:
    api_key = os.getenv("OPENAI_API_KEY", "").strip()
    if not api_key:
        return "Nema OPENAI_API_KEY, a USE_LOCAL=0. Postavi ključ ili prebaci USE_LOCAL=1."

    from openai import OpenAI
    client = OpenAI(api_key=api_key)

    system = build_system_prompt(task_ctx)

    resp = client.chat.completions.create(
        model=OPENAI_MODEL,
        messages=[{"role": "system", "content": system}] + messages,
        temperature=0.2,
    )
    return resp.choices[0].message.content

def call_llm(messages: List[Dict], task_ctx: Dict) -> str:
    if USE_LOCAL:
        return call_ollama(messages, task_ctx)
    return call_openai(messages, task_ctx)

```

Bitno: sad koristi `OLLAMA_BASE=http://localhost:11434` i sam slaže endpoint.

Korak 3: Pokreni Streamlit s ispravnim env varovima

PowerShell:

```

$env:USE_LOCAL="1"
$env:OLLAMA_BASE="http://localhost:11434"
$env:OLLAMA_MODEL="llama3.1"
streamlit run app.py

```

CMD:

```

set USE_LOCAL=1
set OLLAMA_BASE=http://localhost:11434
set OLLAMA_MODEL=llama3.1
streamlit run app.py

```

Što očekujem da ćeš sad vidjeti

Kad klikneš "Ne razumijem":

- ili će se pojaviti normalan assistant odgovor
- ili će se pojaviti assistant poruka tipa:
"Greška pri pozivu modela: HTTPError: 404 Client Error ..."
i onda znamo točno što popraviti u 30 sekundi

Još jedna brza provjera (bez curl)

U browser otvori:

- `http://localhost:11434/api/tags`

Ako dobiješ JSON, Ollama je dostupna.

Pošalji mi screenshot nakon ovih izmjena. Ako se pojavi greška u chatu, samo copy pasteaj taj tekst greške ovdje i odmah ti kažem što dalje.

kad kliknem na ne razumijem samo se zasivi i to bude to, nikad ne dobijem odgovor.

To “zasivi i ništa” je tipično kad se Streamlit zaglavi u tom clicku, najčešće zato što se poziv prema Ollami blokira (request visi) ili pukne prije nego što se UI reruna, ali ti to ne vidiš.

Ajmo to riješiti tako da:

1. vidiš u UI-u gdje je zapelo (spinner + status)
2. imaš “Ping Ollama” gumb koji potvrdi da app uopće može doći do Ollame
3. skratimo timeout i prikažemo grešku u chatu

Ispod su copy paste izmjene.

1) U `llm_backends.py` stavi kratke timeoutove i jasan error

Zamijeni timeout u oba Ollama poziva na 30 sekundi i dodaj `timeout` var.

U `llm_backends.py` koristi ovo (samo relevantni dijelovi):

```
OLLAMA_TIMEOUT = int(os.getenv("OLLAMA_TIMEOUT", "30"))
```

Pa u `_ollama_chat` i `_ollama_generate`:

```
r = requests.post(url, json=payload, timeout=OLLAMA_TIMEOUT)
```

I u fallbacku isto.

Ako želiš full sigurnost, dodaj i “raise for status” poruku:

```
r.raise_for_status()
```

To već imaš.

2) U `chat_ui.py` dodaj spinner i status poruku

Zamijeni `push_and_answer` ovim:

```
def push_and_answer(user_text: str, task_ctx: Dict):
    ensure_chat_state()
    st.session_state.chat_messages.append({"role": "user", "content": user_text})

    with st.spinner("Zovem model..."):
        try:
            answer = call_llm(st.session_state.chat_messages, task_ctx)
            if not answer or not str(answer).strip():
                answer = "Dobio sam prazan odgovor od modela."
            st.session_state.chat_messages.append({"role": "assistant", "content": answer})
        except Exception as e:
            st.session_state.chat_messages.append(
                {"role": "assistant", "content": f"Greška pri pozivu modela: {type(e).__name__}: {e}"}
            )
```

Ovo ti garantira da vidiš barem spinner dok čeka, i da dobiješ poruku ako pukne.

3) U app.py dodaj “Ping Ollama” test gumb (bitno)

Negdje iznad chata (u onom chat bloku gdje su gumbi) dodaj ovo:

```
import requests
import os
```

I onda u chat sekciji, prije gumba “Ne razumijem”, dodaj:

```
OLLAMA_BASE = os.getenv("OLLAMA_BASE", "http://localhost:11434")

if st.button("Ping Ollama"):
    try:
        r = requests.get(f"{OLLAMA_BASE}/api/tags", timeout=5)
        st.success(f'Ollama OK: HTTP {r.status_code}')
    except Exception as e:
        st.error(f'Ollama nije dostupna: {type(e).__name__}: {e}')
```

Ako Ping ne radi, jasno je da Streamlit proces ne može doći do Ollame, i zato ti “Ne razumijem” visi.

4) Najčešći razlog zašto “Ping” ne radi, a Ollama radi u terminalu

Ovo su tri najčešća uzroka na Windowsu:

A) Krivi env var u Streamlit procesu

Provjeri da pokrećeš Streamlit u istom terminalu gdje si postavio env var.

PowerShell:

```
$env:USE_LOCAL="1"
$env:OLLAMA_BASE="http://localhost:11434"
$env:OLLAMA_MODEL="llama3.1"
$env:OLLAMA_TIMEOUT="30"
streamlit run app.py
```

B) Ollama je na drugom hostu ili portu

Neke instalacije slušaju na 0.0.0.0 ili imaju drugačiji port, ali u pravilu je 11434. Ping će ti to otkriti.

C) Firewall ili proxy

Rjeđe, ali moguće. Ping će javiti timeout.

5) Ako Ping radi, a “Ne razumijem” i dalje ništa

Onda je problem u modelu ili endpointu, ali sad ćemo imati grešku u chatu, npr. 404 ili KeyError.

U tom slučaju mi samo copy pasteaj tekst greške koji se pojavi u chatu.

6) Još jedan praktičan hint

Kad klikneš “Ne razumijem”, Streamlit mora čekati odgovor prije reruna. Ako modelu treba 60 do 120 sekundi, izgledat će kao da je mrtvo. Zato timeout 30s i spinner.

Napravi ove 3 izmjene i prvo klikni “Ping Ollama”. Reci mi što ispiše (OK ili error), i ako OK, klikni “Ne razumijem” pa mi pošalji tekst koji se pojavi u chatu (odgovor ili greška).